

OneLink Pay for the UXmaxx Hackathon

Executive summary

Final recommendation: build **Invisible Checkout / OneLink Pay with Repeat-Pay Caps**. Concretely, that means a **web-first, Stripe-like crypto checkout** where a merchant creates a payment link, a customer signs in with an **embedded wallet**, sees a **human-readable transaction preview**, pays from **any supported balance**, and the merchant receives a fixed asset on a single **destination EVM chain** with an on-chain receipt. The highest-probability core stack is **Particle Universal Accounts in EIP-7702 mode + Magic embedded wallet**, with **ZeroDev permissions** as a stretch layer for bounded repeat payments after the first successful checkout. This recommendation follows directly from the public UXmaxx rules: the main challenge is explicitly about **Particle Universal Accounts, EIP-7702, chain abstraction, and embedded wallets**, and its scoring is dominated by **UX & Design at 45%**, ahead of Technical Implementation at 25%, Creativity & Ambition at 20%, and Completeness at 10%. ¹

The key strategic point is that **Particle already solves the rail-layer problem**, so the prize-winning move is not to rebuild routing, bridging, or liquidity. It is to turn that rail layer into a **finished product flow**: payment link, embedded onboarding, preview, receipt, merchant status, and then—only if time allows—**Repeat-Pay Caps**. Particle's docs already expose the critical primitives for that product: 7702 mode as the default path, unified balances across supported chains, `createUniversalTransaction()` for cross-chain execution with follow-up contract calls, and a preview object with token changes and fee quotes. That lets you spend hackathon time on the judge-visible experience instead of infra plumbing. ²

The safest production-of-a-demo choice is **Magic over Particle-native auth as the default**. Particle's own verified-wallet page currently lists **Magic, Privy, and Dynamic** as the embedded-wallet providers verified for Universal Accounts in EIP-7702 mode, and explicitly says standard JSON-RPC wallets such as MetaMask and Rabby cannot be used in that mode. Magic also documents the exact EIP-7702 methods you need—`wallet.sign7702Authorization()` and `wallet.send7702Transaction()`—and offers a very fast Next.js bootstrap path. Particle Auth remains interesting as a sponsor-fit spike because Particle's quickstart uses it for UA examples, but the public 7702 verification page does not currently list Particle Auth itself as a verified embedded-wallet provider for 7702. For a hackathon MVP, that makes Magic the safer default and Particle Auth the timeboxed experiment, not the other way around. ³

Assumptions used in this report: team size is unknown; exact hours remaining are unknown; your current stack depth across frontend, smart contracts, and account-abstraction tooling is unknown. The plan below therefore optimizes for a **small team or solo builder with roughly 48 focused hours**, and treats partner-specific interpretation questions as unknown unless confirmed directly by organizers.

Hackathon constraints

The public rules strongly favor a **checkout product** over a broad “AI agent” or “new wallet infra” pitch. The official page frames UXmaxx around building crypto products that feel **clear, fast, and consumer-ready**,

and the judging weights favor a demo that a non-crypto user can understand immediately. Particle's UA SDK is also currently **Web-only**, which aligns perfectly with a **web-first checkout** rather than mobile or desktop-first scope. ⁴

Checklist item	Official requirement or fact	Implication for OneLink Pay
Qualification mode	Must use Universal Accounts SDK in EIP-7702 mode. ⁵	Build the qualifying path on 7702 mode, not Smart Account mode.
Wallet requirement	Use an embedded wallet like Magic or Openfort, or managed server-side. ⁶	External-wallet-first UX is the wrong primary flow.
Partner technology	Include partner technologies, e.g. ZeroDev. ⁶	Repeat-Pay Caps is the best partner-tech stretch.
Cross-chain proof	Include at least one cross-chain operation moving value via UA. ⁶	Your demo must show actual value sourced from another chain, not just same-chain contract interaction.
Demo requirement	Functional demo must be deployed or runnable locally. ⁶	README, <code>.env.example</code> , and a reliable local run path are mandatory.
Dominant judging axis	UX & Design 45%. ⁷	The preview screen and checkout clarity matter more than adding extra protocol logic.
Technical depth axis	Technical Implementation 25%. ⁷	Correct 7702 handling, clean integration, and edge-case control still matter a lot.
Novelty axis	Creativity & Ambition 20%. ⁷	Repeat-Pay Caps is the best stretch because it is legible and genuinely new.
Completeness axis	Completeness 10%. ⁶	End-to-end flow beats a wider feature list.
Platform scope	UA SDK is currently Web-only. ⁸	Stay web-first; do not spend time on mobile wrappers.

The product is compelling to judges **only if it is clearly more than “a Particle demo.”** The differentiator is not “we also did seamless cross-chain transactions”; Particle already does that. The differentiator is that OneLink Pay turns Particle's rail layer into a **merchant-grade product flow**: one payment link, embedded onboarding, understandable preview, on-chain proof, and bounded follow-up permissions. That is exactly where the public scoring rubric points: non-crypto usability first, then correctness, then ambition. ⁹

Implementation options

The comparison below is **an implementation judgment, not an official ranking.** The factual inputs behind it are public and current: Particle's verified 7702 wallet page lists **Magic, Privy, and Dynamic**; Particle's quickstart uses **Particle Auth**; Magic exposes `sign7702Authorization()` and

`send7702Transaction()` ; Privy supports EIP-7702 authorization and embedded-wallet creation; ZeroDev supports EIP-7702, permissions, and integrations with Magic, Privy, and Particle; Openfort offers embedded wallets, policies, subscriptions, and 7702-related infrastructure but is **not** listed on Particle's verified 7702 wallet page. ¹⁰

Option	Sponsor-fit	Ease of integration	7702 support	Web quickstart	Security tradeoffs	Time-to-MVP	Verdict
Particle Auth/AuthKit + Particle UA	Highest	Medium	Ambiguous in public docs: Particle quickstart uses Particle Auth, but Particle's verified 7702 wallet list does not name it	Strong Particle quickstart	Fewer vendors, but qualifying-path ambiguity is real	Medium	Best timeboxed spike , not best default
Magic + Particle UA	High	Fast	Explicitly verified by Particle	Excellent: Magic CLI + Particle Magic demo	Magic 7702 is headless; your preview screen must do the trust-building work	Fastest	Best default
Magic + Particle UA + ZeroDev	Very high	Medium	Verified for core flow; extra delegation logic adds surface area	Good, but more moving parts	Strong repeat-pay story, but session-key policies must be tightly bounded	Medium	Best stretch-enhanced version

Option	Sponsor-fit	Ease of integration	7702 support	Web quickstart	Security tradeoffs	Time-to-MVP	Verdict
Privy + Particle UA	High	Fast	Explicitly supported by Privy and verified by Particle	Strong React quickstart	Great policies/signers; custom whitelabel flows make auto-wallet creation less automatic	Fast	Best fallback if you already know Privy
Privy + Particle UA + ZeroDev	Very high	Medium	Strong, but stack becomes heavier	Good	Excellent for advanced delegated actions; more debugging paths	Medium	Strong if repeat-pay is central
Openfort + Particle UA	High on paper	Slowest	Openfort is strong on 7702/ payments, but not currently on Particle's verified-wallet page	Openfort web SDK exists, but no Particle-verified bridge path is public	Broad policy surface and more integration uncertainty under hackathon time pressure	Slow	Interesting, but not the default build

Recommended stack decision:

Use **Magic + Particle** for the core MVP; keep **ZeroDev** as the only stretch worth chasing; keep **Privy + Particle** as the fallback branch if your team already has Privy muscle memory or if delegated signers become central; timebox **Particle-native auth** as a short spike only; do **not** choose Openfort as the default path unless you already have working Openfort code and still want to gamble on lower Particle-specific certainty. Particle's verified-wallet page and Magic's 7702 methods make this the cleanest choice under hackathon constraints. ¹¹

MVP architecture

The best build is **OneLink Pay as an EVM-destination checkout**. The merchant should receive **USDC on Base** or another EVM chain because Particle's docs already show `createUniversalTransaction()` funding exact assets on Base and then executing a follow-up contract call there. That gives you the simplest path to a small Solidity **ReceiptEmitter** contract, explorer-friendly proof, and easy event-based merchant

status updates. The **source funds** should come from a different chain or token, so the judges still see real chain abstraction in action. Particle supports a wide set of networks and primary assets, including Base, Arbitrum, Ethereum, and Solana. ¹²

```

flowchart LR
    U[Customer] --> W[Magic embedded wallet]
    W --> UA[Particle Universal Account in EIP-7702 mode]
    UA --> P[Transaction preview and confirmation]
    P --> X[Cross-chain conversion via createUniversalTransaction]
    X --> R[ReceiptEmitter on Base]
    R --> D[Merchant dashboard shows PAID]

```

This architecture is attractive because it expresses a judge-friendly story in one loop: the user logs in like a Web2 app, the app shows a precise payment preview, Particle handles the cross-chain conversion under the hood, and the merchant gets an EVM-side receipt and paid status. That is far stronger than an infra-centric “look, a cross-chain transaction” demo because it packages the capability as a real product. ⁹

User flow

1. Merchant creates an invoice link: product name, amount, destination chain, destination token, merchant payout address.
2. Customer opens `/pay/[invoiceId]`.
3. Customer signs in with Magic email OTP or Magic login UI.
4. App initializes Particle UA in **7702 mode**, fetches **unified balance** with `getPrimaryAssets()`, and fetches account addresses with `getSmartAccountOptions()`.
5. App builds a transaction via `createUniversalTransaction()` with `expectTokens` set to the merchant's payout asset on the destination chain and a follow-up contract call sequence to the receipt contract.
6. App renders a preview using the transaction object's **sender/receiver fields**, **token changes**, and **feeQuotes**.
7. User clicks **Pay**. The app handles any needed 7702 authorization, signs the transaction `rootHash`, and calls `sendTransaction()`.
8. Success page shows paid status, explorer link, receipt ID, and optionally a prompt to enable **Repeat-Pay Caps**. ¹³

Frontend / backend / contract split

Layer	Responsibility	Recommended implementation
Frontend	Merchant form, pay page, preview screen, success page, dashboard	Next.js App Router + TypeScript
Wallet/auth	Embedded login and EOA control	Magic embedded wallet
Chain abstraction	Unified balance, 7702 mode, cross-chain execution	Particle UA SDK

Layer	Responsibility	Recommended implementation
Backend	Invoice CRUD, payment status persistence, basic auth for merchant dashboard	Next.js Route Handlers or minimal Supabase functions
Database	Invoice storage and payment attempts	Supabase/Postgres for deployed MVP; SQLite only as local fallback
Contract	On-chain proof and anti-double-pay state	Minimal <code>ReceiptEmitter</code> on Base

What not to build in v1

Do **not** build a custom bridge, swap router, or in-app quote engine. Particle's product position is explicitly "one account, one balance, any chain," with automatic routing, bridging, and gas abstraction. Re-implementing that stack would burn time in the least judge-visible layer and create new failure modes. Also do **not** make Solana the receipt chain in v1; keep the receipt on Base or another EVM chain for easy proof and simpler contracts. ¹⁴

Data schema

Table	Core fields	Why it exists
<code>invoices</code>	<code>id</code> , <code>merchant_name</code> , <code>merchant_address</code> , <code>destination_chain_id</code> , <code>destination_token_address</code> , <code>token_symbol</code> , <code>amount_atomic</code> , <code>amount_display</code> , <code>currency_label</code> , <code>status</code> , <code>expires_at</code> , <code>created_at</code>	Canonical checkout intent
<code>payment_attempts</code>	<code>id</code> , <code>invoice_id</code> , <code>payer_address</code> , <code>particle_transaction_id</code> , <code>source_balance_snapshot_json</code> , <code>preview_json</code> , <code>status</code> , <code>tx_hash</code> , <code>created_at</code>	Demo visibility, debugability, retry handling
<code>repeat_pay_caps</code>	<code>id</code> , <code>merchant_address</code> , <code>invoice_scope</code> , <code>max_amount_atomic</code> , <code>interval</code> , <code>valid_until</code> , <code>policy_json</code> , <code>status</code>	Stretch feature state for capped future payments

Exact API calls and method usage

Purpose	API or method	Role in MVP
Web login	<code>magic.auth.loginWithEmailOTP()</code> or <code>magic.wallet.connectWithUI()</code>	Start embedded-wallet flow
Wallet metadata	<code>magic.user.getInfo()</code>	Get the user's EVM address

Purpose	API or method	Role in MVP
UA init	<code>new UniversalAccount({ ..., smartAccountOptions: { useEIP7702: true, ... } })</code>	Put checkout on the qualifying execution path
Unified balance	<code>ua.getPrimaryAssets()</code>	Show “You can pay from your available balance”
Account addresses	<code>ua.getSmartAccountOptions()</code>	Display diagnostic or hidden support data
Build preview + route	<code>ua.createUniversalTransaction({ chainId, expectTokens, transactions })</code>	Core payment plan
Read preview	<code>transaction.feeQuotes</code> , <code>transaction.tokenChanges</code> , <code>transaction.sender</code> , <code>transaction.receiver</code>	Human-readable confirmation screen
Check existing delegations	<code>ua.getEIP7702Deployments()</code>	Optional preflight
Fetch auth payload	<code>ua.getEIP7702Auth([chainId])</code>	Optional explicit 7702 prep
Sign 7702 auth	<code>magic.wallet.sign7702Authorization()</code>	Provider-specific authorization signature
Send Type-4 tx	<code>magic.wallet.send7702Transaction()</code>	Provider-level Type-4 path when needed
Sign UA payload	<code>signer.signMessage(transaction.rootHash)</code>	Standard UA signature step
Final broadcast	<code>ua.sendTransaction(transaction, signature, authorizations?)</code>	Execute the routed payment

Particle’s own 7702 wallet page matters here because it documents **both** the provider-agnostic delegation flow and an **inline-authorization** pattern that iterates through `transaction.userOps` and signs pending `eip7702Auth` entries before calling `sendTransaction()`. Magic’s docs, meanwhile, document `sign7702Authorization()` and `send7702Transaction()` directly. In practice, that means you should **start from Particle’s official Magic demo repo** and keep your dependency versions pinned; 7702 wiring is one of the few places where hackathon teams get dragged into avoidable version drift. ¹⁵

Repeat-Pay Caps with ZeroDev

ZeroDev's permissions model matches your stretch feature unusually well. Its documentation frames permissions around **who, when, and what**, and its policy pages say you can restrict **target contracts, allowed functions, argument values**, and **recurring rate limits with reset**—exactly the ingredients for “allow this merchant to charge up to X per day until date Y.” ZeroDev also supports enforcing paymaster usage so a delegated signer cannot grief the user with arbitrary gas choices. ¹⁶

A clean Repeat-Pay Caps policy shape is:

Permission dimension	Concrete rule
Who	A dedicated ZeroDev signer or session key associated with this merchant relationship
When	Valid immediately, expires at <code>validUntil = Friday 23:59 UTC</code> , optionally starts only after first payment
What	Allow only calls to <code>ReceiptEmitter.payInvoice(bytes32)</code> or a tightly scoped checkout contract
How much	Rate-limit-with-reset for a recurring budget, e.g. max <code>\$20/day</code> equivalent
Gas safety	Force a paymaster or a specific paymaster, not arbitrary user-funded gas
Revocation	User-facing toggle in dashboard to disable the cap

That is why Repeat-Pay Caps is the right stretch: it is both **technically real** and **easy for judges to understand**. ¹⁷

Build blueprint

The correct 48-hour strategy is to **de-risk the qualifying transaction first**, then wrap it in a product shell. The public challenge does not reward the broadest scope; it rewards the clearest, complete flow. So your very first milestone is not a dashboard, a landing page, or session keys. It is simply this: **embedded login → UA init in 7702 mode → unified balance → `createUniversalTransaction()` → preview → one successful cross-chain payment → on-chain receipt**. That loop directly covers the challenge's technical requirements and most of its scoring criteria. ⁵

Time block	Goal	Deliverable
Hours zero to four	Create repo, lock architecture, pin versions	Monorepo or app repo, README, <code>.env.example</code> , task board, chosen chains/tokens
Hours four to eight	Prove wallet + UA integration	Magic login works, address visible, UA instance initialized, balance read
Hours eight to twelve	Prove qualifying transaction	One real <code>createUniversalTransaction()</code> succeeds with cross-chain value movement

Time block	Goal	Deliverable
Hours twelve to sixteen	Add preview and confirmation UX	Checkout page shows sender/receiver/token changes/fees before pay
Hours sixteen to twenty-four	Add invoice shell	Merchant can create one invoice; customer can open <code>/pay/[invoiceId]</code>
Hours twenty-four to thirty-two	Add on-chain proof	Deploy <code>ReceiptEmitter</code> , wire follow-up call, show paid status
Hours thirty-two to forty	Demo hardening	Seeded test wallets, success page, explorer link, merchant dashboard, fallback script
Hours forty to forty-eight	Stretch only if core is stable	Repeat-Pay Caps, refined visuals, submission copy, demo video fallback

What to mock

Mock anything that is not part of the scoring-critical story: merchant account systems, catalog management, analytics, tax/VAT logic, invoice emails, settlement reporting, route visuals beyond the preview summary, and any secondary AI/copilot ideas. If you need to fake something, fake the product catalog—not the payment itself. The critical non-mock items are: embedded login, one real cross-chain value-moving operation, preview screen, on-chain proof, and paid status. ⁵

Critical demo items

The shortest prize-worthy demo has five moving parts: a merchant creates one invoice; a customer logs in via embedded wallet; the customer sees a clean preview; the payment succeeds from a non-destination balance; and the merchant dashboard flips to **PAID** with an explorer-proof receipt. If that works flawlessly, you already have a strong submission. Repeat-Pay Caps is only worth adding after that. ¹⁸

Submission checklist

Item	Why it matters
Public URL or reproducible local run path	Required by the challenge
<code>README</code> with architecture diagram and exact stack	Helps technical judges evaluate correctness
<code>.env.example</code> with every required variable named	Reduces reviewer friction
One-paragraph explanation of the cross-chain action	Makes the value movement legible
Short demo video or GIF fallback	Saves you if live routing fails
Explicit mention of EIP-7702 mode and embedded wallet	Shows requirements compliance
Mention ZeroDev only if actually integrated	Avoids overstating partner usage

Item	Why it matters
Explorer link or receipt event screenshot	Gives judges confidence it is real

Contract and frontend skeleton

The contract should stay deliberately small. Particle's own docs show `createUniversalTransaction()` ensuring exact assets on a destination EVM chain and then executing follow-up contract interactions there. So your contract should do **proof and anti-double-pay**, not routing. Ethereum.org's 7702 guidance also matters here: `tx.origin` is no longer safe as an EOA check, reentrancy assumptions must be explicit, and initialization/delegation logic should be kept minimal and audited. ¹⁹

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

contract ReceiptEmitter is Ownable, ReentrancyGuard {
    using SafeERC20 for IERC20;

    struct Invoice {
        address merchant;
        address token;
        uint256 amount;
        uint64 deadline;
        bool exists;
        bool paid;
    }

    mapping(bytes32 => Invoice) public invoices;

    event InvoiceRegistered(
        bytes32 indexed invoiceId,
        address indexed merchant,
        address indexed token,
        uint256 amount,
        uint64 deadline
    );

    event InvoicePaid(
        bytes32 indexed invoiceId,
        address indexed payer,
        address indexed merchant,
```

```

        address token,
        uint256 amount,
        uint256 paidAt,
        uint256 chainId
    );

    error InvoiceAlreadyExists();
    error InvoiceNotFound();
    error InvoiceAlreadyPaid();
    error InvoiceExpired();
    error ZeroAddress();
    error ZeroAmount();

    constructor(address admin) Ownable(admin) {
        if (admin == address(0)) revert ZeroAddress();
    }

    function registerInvoice(
        bytes32 invoiceId,
        address merchant,
        address token,
        uint256 amount,
        uint64 deadline
    ) external onlyOwner {
        if (merchant == address(0) || token == address(0)) revert ZeroAddress();
        if (amount == 0) revert ZeroAmount();
        if (invoices[invoiceId].exists) revert InvoiceAlreadyExists();

        invoices[invoiceId] = Invoice({
            merchant: merchant,
            token: token,
            amount: amount,
            deadline: deadline,
            exists: true,
            paid: false
        });

        emit InvoiceRegistered(invoiceId, merchant, token, amount, deadline);
    }

    function payInvoice(bytes32 invoiceId) external nonReentrant {
        Invoice storage inv = invoices[invoiceId];

        if (!inv.exists) revert InvoiceNotFound();
        if (inv.paid) revert InvoiceAlreadyPaid();
        if (inv.deadline != 0 && block.timestamp > inv.deadline) revert
InvoiceExpired();
    }

```

```

// Checks-effects-interactions
inv.paid = true;

IERC20(inv.token).safeTransferFrom(msg.sender, inv.merchant,
inv.amount);

emit InvoicePaid(
    invoiceId,
    msg.sender,
    inv.merchant,
    inv.token,
    inv.amount,
    block.timestamp,
    block.chainid
);
}

function isPaid(bytes32 invoiceId) external view returns (bool) {
    return invoices[invoiceId].paid;
}

function getInvoice(bytes32 invoiceId) external view returns (Invoice
memory) {
    return invoices[invoiceId];
}
}

```

This skeleton assumes an **approve** → `payInvoice()` pattern. In the Particle transaction, that becomes two follow-up destination-chain calls after the SDK has made the required token available: first `approve(receiptEmitter, amount)`, then `payInvoice(invoiceId)`. That keeps your contract simple while still delivering an obvious on-chain receipt and preventing double payment. For a second version, replace `registerInvoice()` with **EIP-712 merchant-signed invoices** to reduce backend trust and allow permissionless checkout creation. ²⁰

Minimal test set

Test	What it proves
<code>registerInvoice_sets_invoice_fields</code>	Correct storage and event emission
<code>registerInvoice_onlyOwner</code>	Basic access control
<code>payInvoice_transfers_exact_amount_and_emits_event</code>	Happy path works
<code>payInvoice_reverts_if_already_paid</code>	Double-pay prevention
<code>payInvoice_reverts_if_invoice_missing_or_expired</code>	Replay and stale-link protection

Test	What it proves
<code>isPaid_returns_true_after_payment</code>	Dashboard reconciliation path

The frontend should be organized so the **wallet layer is replaceable**. That means a small adapter for Magic now, with a path to Privy later if needed, while everything invoice-related stays unchanged.

```

app/
  api/
    invoices/route.ts
    invoices/[id]/route.ts
  merchant/new/page.tsx
  pay/[id]/page.tsx
  success/[id]/page.tsx
  components/
    LoginCard.tsx
    CheckoutPreviewCard.tsx
    PayButton.tsx
    PaidReceiptCard.tsx
  lib/
    magic.ts
    particle.ts
    wallet-adapter.ts
    receipt.ts
    db.ts
    env.ts
  contracts/
    ReceiptEmitter.sol

```

The snippet below follows Magic's embedded-wallet login APIs and Particle's documented UA initialization and preview flow. It requires real values for `NEXT_PUBLIC_MAGIC_API_KEY`, `NEXT_PUBLIC_PROJECT_ID`, `NEXT_PUBLIC_CLIENT_KEY`, `NEXT_PUBLIC_APP_ID`, a destination RPC, token addresses, and the deployed `ReceiptEmitter` address. ²¹

```

// lib/magic.ts
import { Magic } from "magic-sdk";

export const magic =
  typeof window !== "undefined"
    ? new Magic(process.env.NEXT_PUBLIC_MAGIC_API_KEY!, {
        network: {
          rpcUrl: process.env.NEXT_PUBLIC_BASE_RPC_URL!,
          chainId: Number(process.env.NEXT_PUBLIC_BASE_CHAIN_ID!), // e.g. 8453
        },
      })
    : null

```

```

    : null;

export async function loginWithEmail(email: string) {
  if (!magic) throw new Error("Magic not initialized");
  await magic.auth.loginWithEmailOTP({ email, showUI: true });
  const info = await magic.user.getInfo();
  const address = info.wallets?.ethereum?.publicAddress;
  if (!address) throw new Error("No EVM wallet found");
  return { address };
}

```

```

// lib/particle.ts
import {
  CHAIN_ID,
  SUPPORTED_TOKEN_TYPE,
  UNIVERSAL_ACCOUNT_VERSION,
  UniversalAccount,
} from "@particle-network/universal-account-sdk";
import { BrowserProvider, Interface, Signature, getBytes } from "ethers";
import { magic } from "../magic";

export function createUa(ownerAddress: `0x${string}`) {
  return new UniversalAccount({
    projectId: process.env.NEXT_PUBLIC_PROJECT_ID!,
    projectClientKey: process.env.NEXT_PUBLIC_CLIENT_KEY!,
    projectAppUuid: process.env.NEXT_PUBLIC_APP_ID!,
    smartAccountOptions: {
      useEIP7702: true,
      name: "UNIVERSAL",
      version: UNIVERSAL_ACCOUNT_VERSION,
      ownerAddress,
    },
  },
  {
    tradeConfig: {
      slippageBps: 100,
    },
  });
}

export async function buildInvoiceTransaction(params: {
  ua: UniversalAccount;
  invoiceId: `0x${string}`;
  amountDisplay: string; // human-readable, e.g. "10"
  amountAtomic: bigint; // e.g. 10_000_000n for 10 USDC with 6
  decimals: number;
  destinationChainId?: number; // default Base
  tokenAddress: `0x${string}`; // e.g. Base USDC
}) {

```

```

    receiptEmitter: `0x${string}`;
  }) {
    const erc20 = new Interface([
      "function approve(address spender, uint256 amount) returns (bool)",
    ]);

    const receipt = new Interface([
      "function payInvoice(bytes32 invoiceId)",
    ]);

    return params.ua.createUniversalTransaction({
      chainId: params.destinationChainId ?? CHAIN_ID.BASE_MAINNET,
      expectTokens: [
        {
          type: SUPPORTED_TOKEN_TYPE.USDC,
          amount: params.amountDisplay,
        },
      ],
      transactions: [
        {
          to: params.tokenAddress,
          data: erc20.encodeFunctionData("approve", [
            params.receiptEmitter,
            params.amountAtomic,
          ]),
        },
        {
          to: params.receiptEmitter,
          data: receipt.encodeFunctionData("payInvoice", [params.invoiceId]),
        },
      ],
    });
  }

export async function build7702Authorizations(transaction: any) {
  if (!magic) throw new Error("Magic not initialized");

  const authorizations: { userOpHash: string; signature: string }[] = [];
  const seen = new Map<string, string>();

  for (const userOp of transaction.userOps ?? []) {
    if (userOp.eip7702Auth && !userOp.eip7702Delegated) {
      const key = `${userOp.eip7702Auth.chainId}:${userOp.eip7702Auth.nonce}`;
      let serialized = seen.get(key);

      if (!serialized) {
        const raw = await magic.wallet.sign7702Authorization({
          contractAddress: userOp.eip7702Auth.address,

```

```

        chainId: Number(userOp.eip7702Auth.chainId),
        nonce: Number(userOp.eip7702Auth.nonce),
    });

    serialized = Signature.from({
        r: raw.r,
        s: raw.s,
        v: raw.v,
    }).serialized;

    seen.set(key, serialized);
}

authorizations.push({
    userOpHash: userOp.userOpHash,
    signature: serialized,
});
}
}

return authorizations;
}

export async function sendInvoiceTransaction(ua: UniversalAccount, transaction:
any) {
    if (!magic) throw new Error("Magic not initialized");

    const provider = new BrowserProvider((magic as any).rpcProvider);
    const signer = await provider.getSigner();

    const authorizations = await build7702Authorizations(transaction);
    const rootHashSignature = await
signer.signMessage(getBytes(transaction.rootHash));

    return ua.sendTransaction(transaction, rootHashSignature, authorizations);
}

```

The preview screen should use Particle's returned preview fields rather than hand-built summaries. Particle's preview guide explicitly surfaces `tokenChanges`, `feeQuotes`, sender/receiver fields, and a richer widget design. That should become your confirmation UI, because Magic notes that 7702 can operate **headlessly with no confirmation prompt**. In other words, the preview is not cosmetic; it is part of the safety model. ²²

```

// components/CheckoutPreviewCard.tsx
type Props = { transaction: any };

```



```

export function CheckoutPreviewCard({ transaction }: Props) {
  const feeQuote = transaction?.feeQuotes?.[0];
  const totals = feeQuote?.fees?.totals;
  const decr = transaction?.tokenChanges?.decr ?? [];
  const incr = transaction?.tokenChanges?.incr ?? [];

  return (
    <div className="rounded-xl border p-4 space-y-3">
      <h3 className="text-lg font-semibold">Confirm payment</h3>

      <div>
        <div className="text-sm text-neutral-500">You pay</div>
        <pre className="text-sm overflow-auto">{JSON.stringify(decr, null, 2)}</
pre>
      </div>

      <div>
        <div className="text-sm text-neutral-500">Merchant receives</div>
        <pre className="text-sm overflow-auto">{JSON.stringify(incr, null, 2)}</
pre>
      </div>

      <div className="grid grid-cols-2 gap-3 text-sm">
        <div>
          <div className="text-neutral-500">Sender</div>
          <div className="truncate">{transaction?.sender}</div>
        </div>
        <div>
          <div className="text-neutral-500">Receiver</div>
          <div className="truncate">{transaction?.receiver}</div>
        </div>
        <div>
          <div className="text-neutral-500">Estimated total fee</div>
          <div>{totals?.feeTokenAmountInUSD ?? "n/a"}</div>
        </div>
        <div>
          <div className="text-neutral-500">Gas fee</div>
          <div>{totals?.gasFeeTokenAmountInUSD ?? "n/a"}</div>
        </div>
      </div>
    </div>
  );
}

```

Security, demo, and sources

The single biggest technical risk is not the contract; it is **7702 delegation hygiene**. Ethereum.org is explicit that the EOA private key retains control after delegation, `tx.origin` can no longer be trusted as an “EOA only” signal, initialization can be frontrun if done carelessly, storage collisions are possible when changing delegation contracts, and malicious delegation targets can drain funds—especially if the authorization is broad across chains. Particle adds a practical integration constraint on top: 7702 mode requires embedded wallets with authorization support, and Magic adds a product constraint by documenting that its 7702 flow is headless. Those points should shape both your implementation and your pitch. ²³

Attack vector	Why it matters here	Mitigation
Malicious 7702 delegation / phishing	Delegating to the wrong contract can hand account control to untrusted logic; <code>chain_id=0</code> broadens blast radius across chains. ²⁴	Use only wallet-supported / vetted delegation paths, clearly show destination contract and chain in the UI, and never ask users to authorize arbitrary contracts.
Headless confirmation risk	Magic’s 7702 flow has no built-in confirmation prompt. ²⁵	Make the preview screen mandatory and explicit: amount, merchant, destination chain, fee estimate, and invoice ID.
<code>tx.origin</code> / reentrancy assumptions breaking under 7702	Ethereum.org warns <code>msg.sender == tx.origin</code> is no longer a valid safeguard. ²⁴	Use <code>nonReentrant</code> , checks-effects-interactions, and explicit state checks in the receipt contract.
Initialization frontrunning	7702 init flows can be frontrun if custom initialization is unsafe. ²⁶	Avoid designing your own 7702 delegator for the hackathon; stay inside vendor-supported wallet flows and keep custom contracts off the delegation path.
Storage collision on delegation upgrades	Ethereum’s guidance highlights stale storage when switching delegation contracts. ²⁴	Do not build a custom upgradeable delegator in the MVP.
Invoice replay / double-pay	Same invoice link could be paid twice if state is only off-chain.	Store <code>paid</code> state on-chain or in a signed-intent scheme; use <code>invoiceId</code> , deadline, and exact amount.
Quote drift / oracle-style UI ambiguity	Particle preview surfaces estimates, not a legal guarantee of a live USD rate. ²⁷	Fix settlement in exact token units on the destination chain; never let UI USD estimates change the invoice amount.

Attack vector	Why it matters here	Mitigation
Session-key overspend	Delegated repeat-pay logic can become a blank check if unconstrained. ¹⁶	Use target allowlists, function restrictions, max spend per interval, expiry, and paymaster enforcement.
Wallet-path ambiguity	Particle's quickstart uses Particle Auth, but its verified 7702 wallet list publicly names Magic, Privy, and Dynamic. ²⁸	Keep a wallet-adapter boundary and choose Magic as the default qualifying path.
Docs drift / UA V2 ambiguity	Particle currently shows both a V2 migration notice and older FAQ language about deposits into UA smart accounts. ²⁹	Start from the latest official sample repo, pin versions, and prove the exact chain pair early.

Demo script

Open on the merchant side first. Create a payment link for a simple product such as “UXmaxx Hoodie — 10 USDC on Base.” Then switch to the customer view and open that link. Log in with email using the embedded wallet. Show the preview screen and say, in one sentence, that the user’s spendable balance can live anywhere Particle supports and the merchant still receives the exact asset on Base. Click **Pay**. Show the receipt event or explorer link and then switch back to the merchant dashboard to show **PAID**. If—and only if—the core loop is rock solid, finish with the Repeat-Pay Caps toggle: “Allow future payments to this merchant up to 20 USDC per day until Friday.” Then show one allowed repeat payment and one blocked over-limit attempt. That sequence mirrors the scoring: UX first, then technical proof, then ambition. ³⁰

Thirty-to-sixty-second pitch

OneLink Pay is an invisible crypto checkout built for the UXmaxx challenge. Today, crypto payments still force users to think about chains, gas, bridging, and where their funds sit. With OneLink Pay, a merchant creates one payment link, a user logs in with email through an embedded wallet, sees a clear payment preview, and pays from any supported balance without switching networks or hunting for gas. Under the hood we use Particle Universal Accounts in EIP-7702 mode to execute a real cross-chain payment, and the merchant receives the exact asset on the destination chain with an on-chain receipt. As a stretch, we add Repeat-Pay Caps so users can safely allow small future payments to the same merchant within strict limits.

Submission text

OneLink Pay is a Stripe-like crypto checkout powered by Particle Universal Accounts and EIP-7702. A merchant creates a payment link, and the customer signs in with an embedded wallet, sees a human-readable transaction preview, and pays from any supported balance without switching networks, bridging manually, or managing destination-chain gas. The merchant receives the desired asset on a destination EVM chain, and the app emits an on-chain receipt for proof of payment. The core product demonstrates a real cross-chain value-moving operation via Universal Accounts. As a stretch feature, OneLink Pay

introduces Repeat-Pay Caps: narrowly scoped, revocable future payment permissions with hard limits such as merchant allowlists, daily spend caps, and expiry windows.

Likely judge questions and strong answers

Judge question	Suggested answer
Why is this more than a Particle demo?	Particle solves the infrastructure layer. We turned it into a finished merchant flow: payment links, preview UX, receipt proof, paid status, and bounded future payments.
Why is Web3 necessary here?	The user still controls a wallet, the merchant can settle globally in crypto, and the payment is transparently provable on-chain; the innovation is removing chain fragmentation without removing user ownership.
What is the actual cross-chain operation?	The user's spendable balance is sourced from a different supported chain or asset, while the merchant is paid a fixed asset on the destination chain through Particle UA.
Why Magic instead of a normal wallet?	Particle's public 7702 guidance requires an embedded wallet with authorization support, and Magic is one of the providers Particle explicitly verifies for that path.
How do Repeat-Pay Caps stay safe?	They are not broad approvals; they are constrained permissions with merchant allowlists, amount limits, time windows, and revocation.
What did you cut for scope?	We deliberately did not build our own router, bridge, or quote engine. We used Particle for the rail layer and spent our time on the judge-visible product surface.
What are the MVP limitations?	Single-merchant or limited-merchant demo scope, fixed destination chain in v1, and capped-repeat permissions only as a stretch if the core payment loop is already reliable.

Prioritized official docs and sample repos

If I were implementing this from zero, I would read the sources in this order:

- **UXmaxx official page** for constraints and scoring. ³¹
- **Particle verified 7702 wallets page** for what is actually safe to rely on in an embedded-wallet flow. ³²
- **Particle Web SDK reference** for 7702 mode, `useEIP7702`, unified balances, `getPrimaryAssets()`, `getSmartAccountOptions()`, `createUniversalTransaction()`, and `sendTransaction()`. ³³
- **Particle preview guide** for the checkout confirmation UI. ²⁷
- **Particle quickstart repo** as the baseline implementation skeleton. ³⁴
- **Particle Magic 7702 demo repo** for the exact qualifying wallet path. ³⁵
- **Particle Privy 7702 demo repo** as the strongest fallback branch. ³⁶
- **Magic docs** for Email OTP, embedded-wallet bootstrap, wallet metadata, and 7702 methods. ³⁷

- **Privy docs** for React quickstart, automatic wallet creation caveats, policies, signers, and EIP-7702 authorization. ³⁸
- **ZeroDev docs** for permissions/session keys, call policies, recurring limits, gas rules, and Magic/Privy/Particle integrations. ³⁹
- **Openfort official materials** if you want a payments-first alternative or a future branch. ⁴⁰
- **Ethereum.org and the EIP-7702 spec** for the security model, risk boundaries, and why your preview and delegation hygiene matter. ²⁴

The bottom line is simple: **build Invisible Checkout first, not programmable payments first**. Ship the one-link checkout, human-readable preview, real cross-chain payment, and EVM receipt. Then add **Repeat-Pay Caps** only if the core loop is already boringly reliable. Under the current public rules and the current provider docs, that is the highest expected-value path to a prize. ⁴¹

¹ ⁴ ⁵ ⁶ ⁷ ⁹ ¹⁸ ³⁰ ³¹ ⁴¹ <https://www.encodeclub.com/programmes/uxmaxx-hackathon>
<https://www.encodeclub.com/programmes/uxmaxx-hackathon>

² ³ ¹⁰ ¹¹ ¹⁵ ³² <https://developers.particle.network/universal-accounts/ua-reference/desktop/eip7702-wallets>
<https://developers.particle.network/universal-accounts/ua-reference/desktop/eip7702-wallets>

⁸ <https://developers.particle.network/universal-accounts/ua-reference/faq>
<https://developers.particle.network/universal-accounts/ua-reference/faq>

¹² ¹³ ¹⁹ ²⁰ ³³ <https://developers.particle.network/universal-accounts/ua-reference/desktop/web>
<https://developers.particle.network/universal-accounts/ua-reference/desktop/web>

¹⁴ <https://developers.particle.network/intro/universal-accounts>
<https://developers.particle.network/intro/universal-accounts>

¹⁶ ¹⁷ ³⁹ <https://docs.zerodev.app/sdk/permissions/intro>
<https://docs.zerodev.app/sdk/permissions/intro>

²¹ ³⁷ <https://docs.magic.link/embedded-wallets/authentication/login/email-otp>
<https://docs.magic.link/embedded-wallets/authentication/login/email-otp>

²² ²⁷ <https://developers.particle.network/universal-accounts/cha/how-to/tx-preview>
<https://developers.particle.network/universal-accounts/cha/how-to/tx-preview>

²³ ²⁴ ²⁶ <https://ethereum.org/roadmap/pectra/7702/>
<https://ethereum.org/roadmap/pectra/7702/>

²⁵ <https://docs.magic.link/embedded-wallets/wallets/features/eip-7702>
<https://docs.magic.link/embedded-wallets/wallets/features/eip-7702>

²⁸ ²⁹ <https://developers.particle.network/universal-accounts/cha/web-quickstart>
<https://developers.particle.network/universal-accounts/cha/web-quickstart>

³⁴ <https://github.com/Particle-Network/universal-accounts-quickstart>
<https://github.com/Particle-Network/universal-accounts-quickstart>

³⁵ <https://github.com/Particle-Network/ua-7702-magic-demo>
<https://github.com/Particle-Network/ua-7702-magic-demo>

³⁶ <https://github.com/Particle-Network/universal-accounts-7702>
<https://github.com/Particle-Network/universal-accounts-7702>

³⁸ <https://docs.privy.io/basics/react/quickstart>
<https://docs.privy.io/basics/react/quickstart>

⁴⁰ <https://www.openfort.io/>
<https://www.openfort.io/>